



Obyektga yo'naltirilgan dasturlash (OOP) asoslari

Klass va obyekt, `__init__` va `self`, atributlar va metodlar, `__str__`, inkapsulyatsiya, merosxo'rlik (inheritance), polimorfizm; kutubxona boshqaruv tizimi loyihasi.

2 soat · 1 nazariy + 1 amaliy

Bu darsda nimani o'rganasiz

- **Klass** va **obyekt** nima va ular nima uchun kerakligini;
- `__init__`, `self`, atributlar va metodlarni;
- OOP'ning uch ustuni: **inkapsulyatsiya**, **merosxo'rlik**, **polimorfizm**;
- Kutubxona boshqaruv tizimini (17-loyiha) OOP uslubida yaratishni.

Bu mavzu Django uchun juda muhim: Django'da modellar, formalar va view'lar — hammasi klasslar!

Nima uchun OOP?

Hozirgacha ma'lumotlarni (dict) va ular bilan ishlovchi funksiyalarni **alohida** saqladik. Dastur kattalashganda bu chalkashadi: qaysi funksiya qaysi ma'lumot uchun? **Obyektga yo'naltirilgan dasturlash (OOP)** ma'lumot va unga tegishli amallarni **bitta "obyekt" ichida** birlashtiradi.



O'xshatish: avtomobil zavodi

Klass — avtomobilning **chizmasi (loyihasi)**: unda "rangi bor, tezligi bor, yura oladi, to'xtay oladi" deb yozilgan. Chizmaning o'zi yurmaydi! **Obyekt** — shu chizma asosida yasalgan **haqiqiy mashina**: qizil Cobalt, oq Malibu... Bitta chizmadan minglab mashina yasash mumkin, har birining rangi va probegi o'ziniki, lekin hammasi "yura oladi".

Tushuncha	Ma'nosi	Misol
Klass (class)	Shablon, chizma	<code>0quvchi</code>

Obyekt (object, instance)	Klassdan yaratilgan aniq nusxa	ali, zarina
Atribut (attribute)	Obyektning xususiyati (ma'lumot)	ism, sinf, baholar
Metod (method)	Obyekt bajara oladigan amal (funksiya)	baho_qosh(), ortacha()

Aslida siz OOP bilan allaqachon tanishsiz: `"salom".upper()` — bu `str` klassi obyektining `upper` metodi; `royxat.append(5)` — `list` obyektining metodi.

Birinchi klass

Python

```
class Oquvchi:                                # klass nomi – KattaHarfBilan
(PascalCase)
    def __init__(self, ism, sinf):            # konstruktor: obyekt
varatilganda chaqiriladi
        self.ism = ism                        # atributlar
        self.sinf = sinf
        self.baholar = []                    # har bir o'quvchida o'z bo'sh
ro'yxati

    def baho_qosh(self, baho):                # metod
        self.baholar.append(baho)

    def ortacha(self):
        if not self.baholar:
            return 0
        return round(sum(self.baholar) / len(self.baholar), 2)

    def tanishtir(self):
        print(f"Men {self.ism}, {self.sinf} sinf. O'rtacha bahom:
{self.ortacha()}")

# Obyektlar yaratish
ali = Oquvchi("Ali", "8-A")
zarina = Oquvchi("Zarina", "8-B")

ali.baho_qosh(5)
ali.baho_qosh(4)
zarina.baho_qosh(5)

ali.tanishtir()
zarina.tanishtir()
print(ali.ism, ali.baholar)
```

Natija

```
Men Ali, 8-A sinf. O'rtacha bahom: 4.5
Men Zarina, 8-B sinf. O'rtacha bahom: 5.0
Ali [5, 4]
```

`__init__` va `self` — eng muhim ikki tushuncha

- `__init__` (initialize — boshlang'ich holatga keltirish) — **konstruktor**. `Oquvchi("Ali", "8-A")` yozilganda avtomatik chaqiriladi va yangi obyektning atributlarini "to'ldiradi".
- `self` — "**shu obyektning o'zi**". `ali.baho_qosh(5)` chaqirilganda, metod ichidagi `self` — `ali` bo'ladi; `zarina.baho_qosh(5)` da — `zarina`. Shuning uchun har bir o'quvchining baholari alohida saqlanadi.



`self` — "men"

Sinfdagi har bir o'quvchi "mening ismim", "mening daftarim" deydi. So'z bir xil — "mening", lekin har bir o'quvchi uchun u boshqa narsani bildiradi. `self.ism` — "mening ismim".



Har bir metodning **birinchi parametri** — `self`. Uni yozishni unutsangiz: `TypeError: takes 1 positional argument but 2 were given`. Chaqirganda esa `self` berilmaydi — Python uni o'zi qo'yadi.

`__str__` — obyektни chiroyli chiqarish

Python

```
class Kitob:
    def __init__(self, nomi, muallif, yil):
        self.nomi = nomi
        self.muallif = muallif
        self.yil = yil

    def __str__(self):
        # print(obyekt) chaqirilganda
        return f"«{self.nomi}» — {self.muallif} ({self.yil})"

k = Kitob("O'tkan kunlar", "Abdulla Qodiriy", 1926)
print(k)          # «O'tkan kunlar» — Abdulla Qodiriy (1926)
# __str__ bo'lmaganda: <__main__.Kitob object at 0x7f...>
```

`__init__`, `__str__` kabi ikki pastki chiziq bilan o'ralgan metodlar — "**maxsus (magic, dunder) metodlar**". Django modellarida `__str__` ni albatta yozamiz (36-mavzu) — admin panelda obyektlar shu nom bilan ko'rinadi.

OOP'ning uch ustuni

1. Inkapsulyatsiya — ichki ishlarni yashirish

Obyekt o'z ma'lumotlarini **himoya qiladi** va ularni faqat **o'z metodlari orqali** o'zgartirishga ruxsat beradi.

```

class BankHisobi:
    def __init__(self, egasi, balans=0):
        self.egasi = egasi
        self._balans = balans          # _ - "ichki, tashqaridan tegmang" degan
kelishuv

    def toldirish(self, summa):
        if summa <= 0:
            print("❌ Summa musbat bo'lishi kerak")
            return
        self._balans += summa

    def yechish(self, summa):
        if summa > self._balans:
            print("❌ Mablag' yetarli emas")
            return
        self._balans -= summa

    def balans(self):
        return self._balans

h = BankHisobi("Jasur", 100_000)
h.yechish(500_000)          # ❌ Mablag' yetarli emas
h.toldirish(-50)           # ❌ Summa musbat bo'lishi kerak
h.toldirish(50_000)
print(h.balans())          # 150000

```



O'xshatish: bankomat

Siz bankomatdagi pulni qo'lingiz bilan olmaysiz — faqat tugmalar (metodlar) orqali so'raysiz, bankomat esa qoidalarni tekshiradi (balans yetarlimi, PIN to'g'rimi). Ichki mexanizm yashirin — bu inkapsulyatsiya.

2. Merosxo'rlik (inheritance) — mavjud klassdan yangisini yaratish

Yangi klass boshqa klassning barcha atribut va metodlarini **meros qilib oladi** va o'zinikini qo'shadi.

```

class Odam:                                # ota (asosiy) klass
    def __init__(self, ism, yosh):
        self.ism = ism
        self.yosh = yosh

    def tanishtir(self):
        return f"Men {self.ism}, {self.yosh} yoshdaman"

class Oquvchi(Odam):                        # Odam'dan meros oladi
    def __init__(self, ism, yosh, sinf):
        super().__init__(ism, yosh)        # ota klass konstruktorini
chaqirish
        self.sinf = sinf                  # yangi atribut

    def dars_qil(self):                     # yangi metod
        return f"{self.ism} uy vazifasini bajaryapti 📖"

class Oqituvchi(Odam):
    def __init__(self, ism, yosh, fan):
        super().__init__(ism, yosh)
        self.fan = fan

    def tanishtir(self):                   # ota metodini QAYTA YOZISH
(override)
        return f"Men {self.ism}, {self.fan} fani o'qituvchisiman"

o = Oquvchi("Madina", 14, "8-A")
t = Oqituvchi("Rustam aka", 35, "informatika")
print(o.tanishtir())                     # ota klassdan meros
print(o.dars_qil())                      # o'zining metodi
print(t.tanishtir())                     # qayta yozilgan metod

```

Natija

```

Men Madina, 14 yoshdaman
Madina uy vazifasini bajaryapti 📖
Men Rustam aka, informatika fani o'qituvchisiman

```

Odam

ism, yosh, tanishtir()

→ meros →

Oquvchi

+ sinf, + dars_qil()

Oqituvchi

+ fan, tanishtir() qayta yozilgan

Django'da buni doim ko'rasiz: `class Kitob(models.Model):` — sizning modelingiz Django'ning tayyor `Model` klassidan meros oladi va bazaga saqlash kabi yuzlab imkoniyatlarni "tekinga" oladi.

3. Polimorfizm — bir xil buyruq, turli xatti-harakat

Python

```
class Hayvon:
    def __init__(self, ism):
        self.ism = ism
    def ovoz(self):
        return "... "

class Mushuk(Hayvon):
    def ovoz(self):
        return "Miyov 🐱"

class It(Hayvon):
    def ovoz(self):
        return "Vov-vov 🐶"

class Sigir(Hayvon):
    def ovoz(self):
        return "Mo-o-o 🐮"

ferma = [Mushuk("Mosh"), It("Bobik"), Sigir("Zebo")]
for h in ferma:
    print(f"{h.ism}: {h.ovoiz()}")      # bitta buyruq — har biri o'zicha javob
beradi
```

Polimorfizm (ko'p shakllilik) — turli klass obyektlari bir xil nomli metodga ega bo'lib, har biri uni o'zicha bajaradi. Dastur obyektning aniq turini bilmasa ham ishlay oladi.

Amaliy mashg'ulot: kutubxona boshqaruv tizimi (17-

loyiha)

kutubxona.py

```
from datetime import date, timedelta

class Kitob:
    def __init__(self, isbn, nomi, muallif, nusxa=1):
        self.isbn = isbn
        self.nomi = nomi
        self.muallif = muallif
        self.jami = nusxa
        self.mavjud = nusxa

    def __str__(self):
        return f"[{self.isbn}] «{self.nomi}» – {self.muallif} ({self.mavjud}/{self.jami})"

class Azo:
    MAKS_KITOB = 3 # klass atributi – barcha a'zolar uchun umumiy

    def __init__(self, id, ism):
        self.id = id
        self.ism = ism
        self.kitoblar = {} # isbn → qaytarish muddati

    def __str__(self):
        return f"#{self.id} {self.ism} (qo'lida: {len(self.kitoblar)} ta kitob)"

class Kutubxona:
    IJARA_KUNI = 14
    JARIMA_KUNIGA = 1000

    def __init__(self, nomi):
        self.nomi = nomi
        self.kitoblar = {} # isbn → Kitob
        self.azolar = {} # id → Azo
```

```

def kitob_qosh(self, kitob):
    if kitob.isbn in self.kitoblar:
        mavjud = self.kitoblar[kitob.isbn]
        mavjud.jami += kitob.jami
        mavjud.mavjud += kitob.jami
    else:
        self.kitoblar[kitob.isbn] = kitob

def azo_qosh(self, azo):
    self.azolar[azo.id] = azo

def qidir(self, soz):
    soz = soz.lower()
    return [k for k in self.kitoblar.values()
            if soz in k.nomi.lower() or soz in k.muallif.lower()]

def berish(self, azo_id, isbn, bugun=None):
    bugun = bugun or date.today()
    azo = self.azolar.get(azo_id)
    kitob = self.kitoblar.get(isbn)
    if not azo or not kitob:
        return "❌ A'zo yoki kitob topilmadi"
    if kitob.mavjud == 0:
        return f"❌ «{kitob.nomi}» hozir mavjud emas"
    if len(azo.kitoblar) ≥ Azo.MAKS_KITOB:
        return f"❌ {azo.ism} qo'lida allaqachon {Azo.MAKS_KITOB} ta kitob
bor"

    if isbn in azo.kitoblar:
        return "❌ Bu kitob allaqachon sizda"
    kitob.mavjud -= 1
    muddat = bugun + timedelta(days=self.IJARA_KUNI)
    azo.kitoblar[isbn] = muddat
    return f"✅ «{kitob.nomi}» {azo.ism}ga berildi. Qaytarish:
{muddat:%d.%m.%Y}"

def qaytarish(self, azo_id, isbn, bugun=None):
    bugun = bugun or date.today()
    azo = self.azolar.get(azo_id)
    if not azo or isbn not in azo.kitoblar:
        return "❌ Bunday ijara yo'q"
    muddat = azo.kitoblar.pop(isbn)
    self.kitoblar[isbn].mavjud += 1
    kechikish = (bugun - muddat).days
    if kechikish > 0:

```

```

        return f"⚠️ {kechikish} kun kechikdi. Jarima: {kechikish *
self.JARIMA_KUNIGA:,} so'm"
        return "✅ Kitob o'z vaqtida qaytarildi. Rahmat!"

    def hisobot(self):
        print(f"\n📖 {self.nomi}")
        for k in self.kitoblar.values():
            print("  ", k)
        for a in self.azolar.values():
            print("  ", a)

# ===== Sinov =====
lib = Kutubxona("Maktab kutubxonasi")
lib.kitob_qosh(Kitob("001", "O'tkan kunlar", "Abdulla Qodiriy", 2))
lib.kitob_qosh(Kitob("002", "Sariq devni minib", "Xudoyberdi To'xtaboyev"))
lib.kitob_qosh(Kitob("003", "Shum bola", "G'afur G'ulom"))
lib.azo_qosh(Azo(1, "Aziz"))
lib.azo_qosh(Azo(2, "Nilufar"))

print(lib.berish(1, "002", date(2026, 9, 1)))
print(lib.berish(2, "002")) # mavjud emas
print([str(k) for k in lib.qidir("qodiriy")])
print(lib.qaytarish(1, "002", date(2026, 9, 20))) # 5 kun kechikish
lib.hisobot()

```

Natija

✅ «Sariq devni minib» Azizga berildi. Qaytarish: 15.09.2026
 ❌ «Sariq devni minib» hozir mavjud emas
 ["[001] «O'tkan kunlar» – Abdulla Qodiriy (2/2)"]
 ⚠️ 5 kun kechikdi. Jarima: 5,000 so'm

📖 Maktab kutubxonasi
 [001] «O'tkan kunlar» – Abdulla Qodiriy (2/2)
 [002] «Sariq devni minib» – Xudoyberdi To'xtaboyev (1/1)
 [003] «Shum bola» – G'afur G'ulom (1/1)
 #1 Aziz (qo'lida: 0 ta kitob)
 #2 Nilufar (qo'lida: 0 ta kitob)

Topshiriqlar:

- 1 Kodni qatorma-qator o'qing va har bir klassning vazifasini o'z so'zlaringiz bilan yozing.
- 2 `Azo` dan meros olgan `Oqituvchi` klassini yarating: unda `MAKS_KITOB = 10`. Polimorfizm ishlashini tekshiring.
- 3 Menyu qo'shing (23-mavzudagi kabi): kitob qo'shish, qidirish, berish, qaytarish, hisobot.
- 4 Ma'lumotlarni JSON faylga saqlash va yuklash metodlarini yozing (`saqla()`, `yukla()`).

Ko'p uchraydigan xatolar

- ✗ Metodda `self` parametrini unutish.
- ✗ Metod ichida `ism` deb yozish (`self.ism` o'rniga) — `NameError`.
- ✗ `__init__` ni bitta pastki chiziq bilan yozish (`_init_`) — konstruktor ishlamaydi.
- ✗ Merosxo'r klassda `super().__init__(...)` ni chaqirishni unutish — ota klass atributlari yaratilmaydi.
- ✗ Obyekt yaratishda qavsni unutish: `ali = Oquvchi` — bu klassning o'zi, obyekt emas.

O'zingizni tekshiring

? Klass va obyekt o'rtasidagi munosabat qaysi?

- A. Obyekt — chizma, klass — mahsulot
- B. Klass — chizma (shablon), obyekt — undan yaratilgan nusxa
- C. Ular bir xil narsa
- D. Klass — funksiya, obyekt — o'zgaruvchi

? Obyekt yaratilganda avtomatik chaqiriladigan metod?

- A. `__str__`
- B. `__main__`
- C. `__init__`
- D. `create`

? `self` nimani bildiradi?

- A. Metod chaqirilayotgan obyektning o'zini
- B. Klass nomini
- C. Global o'zgaruvchini
- D. Ota klassni

? `class It(Hayvon):` — bu qaysi OOP tamoyili?

- A. Inkapsulyatsiya
- B. Polimorfizm
- C. Abstraksiya
- D. Merosxo'rlik

? `print(obyekt)` qanday matn chiqarishini qaysi metod belgilaydi?

- A. `__init__`
- B. `__str__`
- C. `__print__`
- D. `show()`

? Turli klasslar bir xil nomli metodni o'zicha bajarishi qanday ataladi?

- A. Inkapsulyatsiya
- B. Merosxo'rlik
- C. Polimorfizm
- D. Kompilyatsiya



Uyga vazifa

1. **Geometrik shakllar:** `Shakl` ota klassi va undan meros olgan `Doira`, `Tortburchak`, `Uchburchak` — har birida `yuza()` va `perimetr()`. Shakllar ro'yxatini aylanib, yuzalarni chiqaring (polimorfizm).
2. **Internet do'kon savati:** `Mahsulot` va `Savat` klasslari: qo'shish, o'chirish, jami summa, chegirma qo'llash.
3. **Sinf jurnali:** `Sinf` klassi ichida `Oquvchi` obyektlari ro'yxati; eng yaxshi o'quvchi, sinf o'rtachasi metodlari.



Yodda tuting

- `class Nom:` — chizma; `obj = Nom(...)` — obyekt.
- `def __init__(self, ...):` — konstruktor; `self.attribut` — obyekt ma'lumoti; metodlar birinchi parametri — `self`.
- `__str__` — obyektning matn ko'rinishi.
- **Inkapsulyatsiya** — ma'lumotni metodlar orqali himoyalash; **merosxo'rlik** — `class B(A): + super().__init__()`; **polimorfizm** — bir xil metod, turli xatti-harakat.

☐ Darsni o'qib bo'ldim

✓ Test javoblari

1. Klass va obyekt o'rtasidagi munosabat qaysi? — **B) Klass — chizma (shablon), obyekt — undan yaratilgan nusxa**
2. Obyekt yaratilganda avtomatik chaqiriladigan metod? — **C) __init__**
3. self nimani bildiradi? — **A) Metod chaqirilayotgan obyektning o'zini**
4. class It(Hayvon): — bu qaysi OOP tamoyili? — **D) Merosxo'rlik**
5. print(obyekt) qanday matn chiqarishini qaysi metod belgilaydi? — **B) __str__**
6. Turli klasslar bir xil nomli metodni o'zicha bajarishi qanday ataladi? — **C) Polimorfizm**